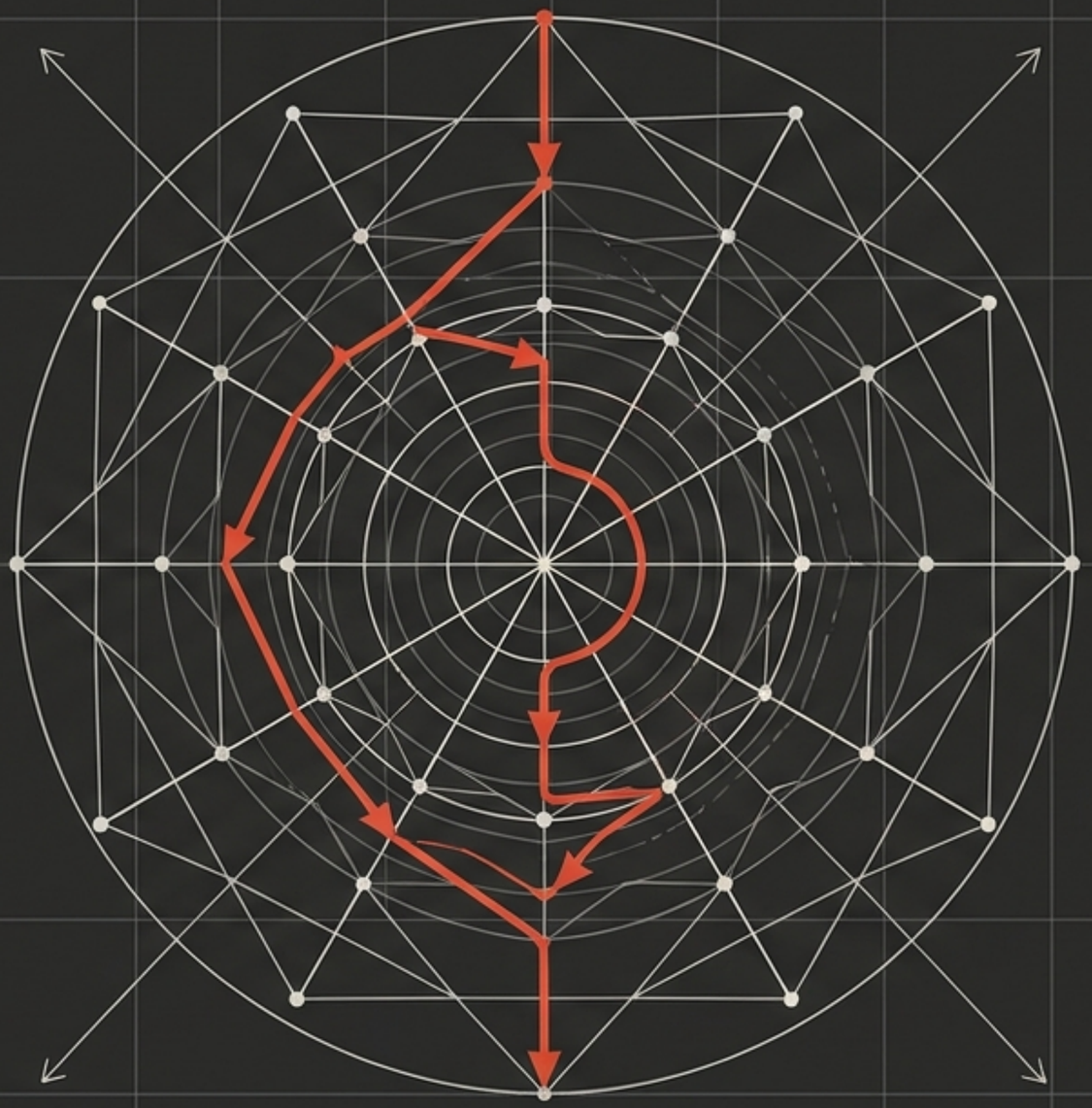
Busca em Profundidade (DFS)

Dominando a Exploração de Grafos: Da Teoria ao Código

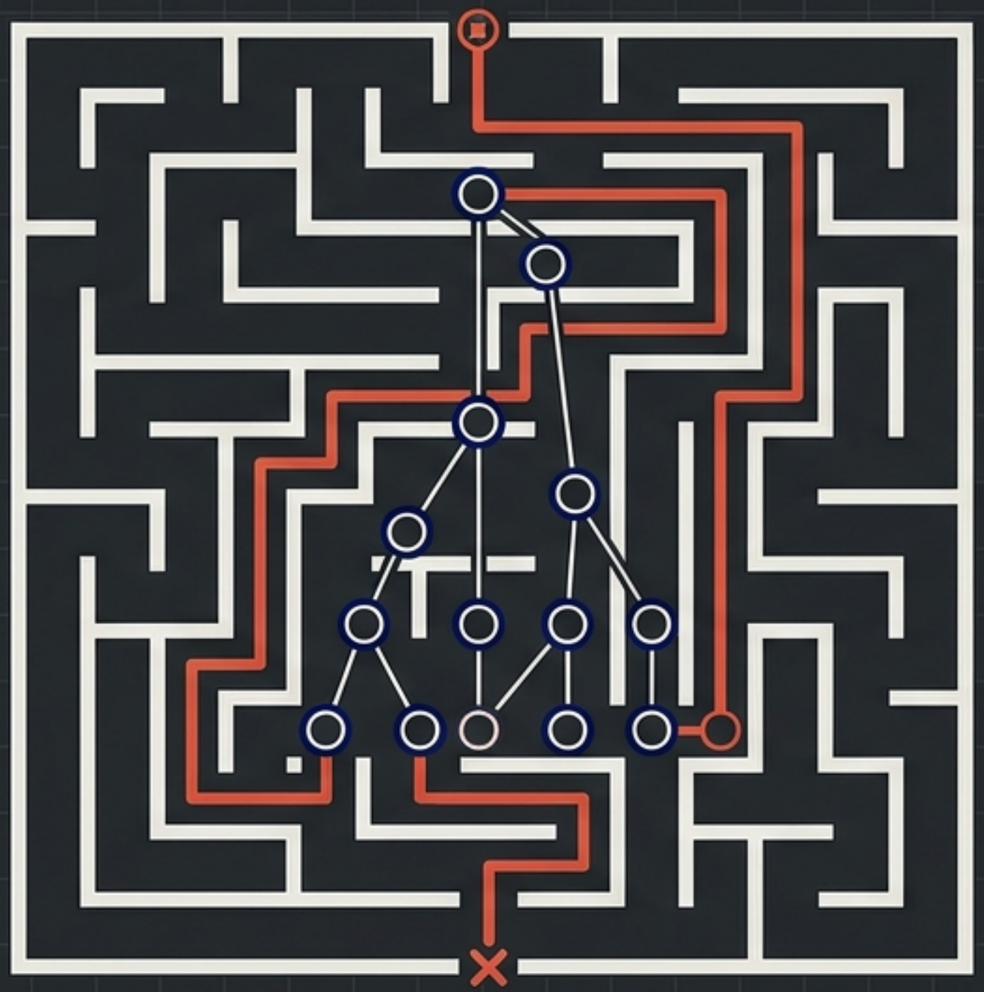
[TAG: Algoritmos em Grafos]	[DURAÇÃO ESTIMADA: Material para 1 Hora de Aula]	[NÍVEL: Intermediário / Avançado]



Gemini Notebook

A Intuição: Explorando o Labirinto

A regra de ouro do DFS é a exploração agressiva: vá o mais fundo possível até não ter mais para onde ir.



O Mergulho

Siga pelo primeiro corredor inexplorado. Priorize a profundidade.



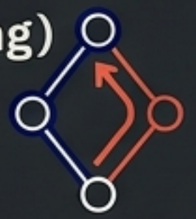
Fim da Linha

Identifique becos sem saída onde não há vizinhos livres.



Retrocesso (Backtracking)

Volte pelo mesmo caminho até achar uma nova rota e mergulhe novamente.



Gemini Notebook

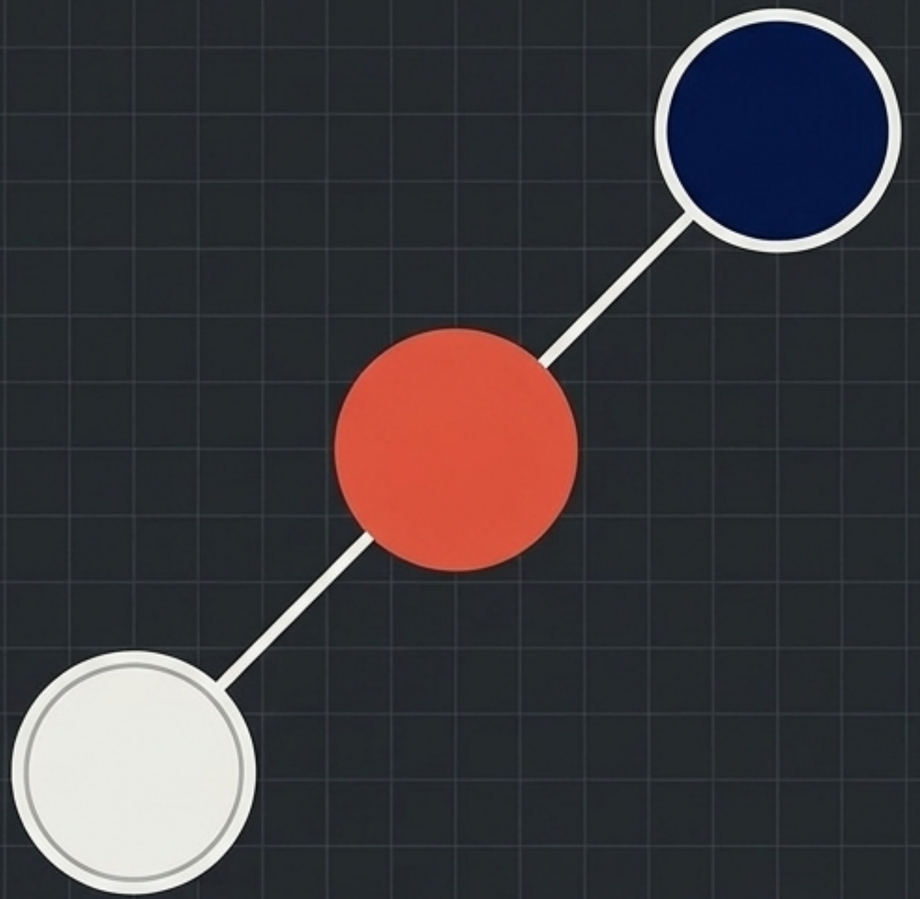
A Anatomia da Busca: A Regra das 3 Cores

O algoritmo rastreia o estado de cada vértice pintando-o à medida que a fronteira avança, evitando loops infinitos.

Branco (Não Visitado) - Território desconhecido. O vértice ainda não foi descoberto.

Vermelho (Na Fronteira) - Descoberto, mas com vizinhos inexplorados. Ativamente na Pilha.

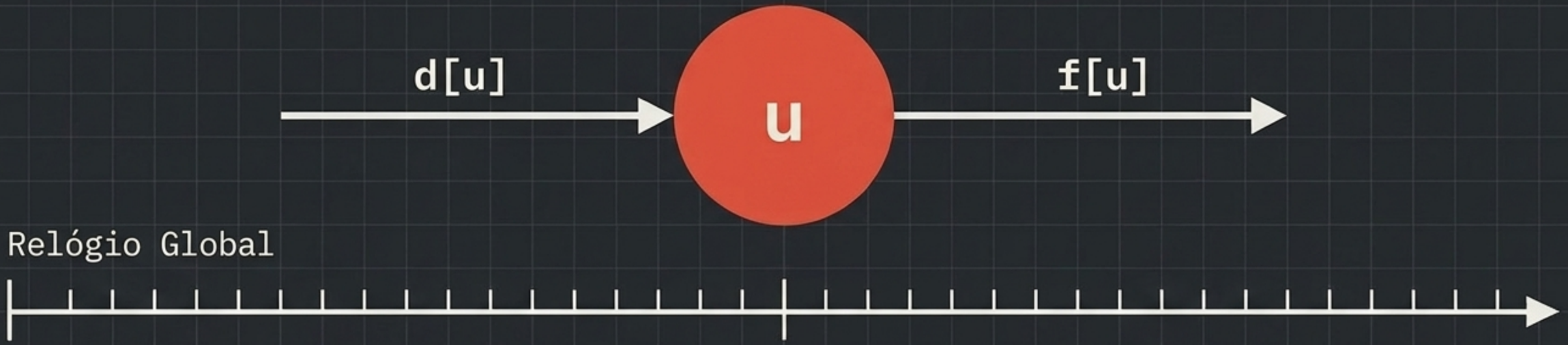
Preto (Finalizado) - Beco sem saída mapeado. Vizinhos processados. Removido da Pilha.



Gemini Notebook

Rastreando o Tempo (Timestamps)

O DFS atrela 'carimbos de tempo' a cada vértice, revelando a estrutura hierárquica do grafo.



Descoberta d[u]

O instante no relógio global em que o vértice torna-se Vermelho (entra na pilha).

Finalização f[u]

O instante no relógio global em que o vértice torna-se Preto (sai da pilha).

Insight de Engenharia: A relação d[u] < f[u] indica que descendentes têm tempos de vida contidos nos de seus ancestrais.

Gemini Notebook

A Máquina Oculta: DFS Recursivo (Python)

```python
def dfs_recursive(tree, node, visited=None):
    dfs_recursive(tree, node, visited=None):
    if visited is None:
        visited = set()

    visited.add(node)

    for child in tree[node]:
        if child not in visited:
            dfs_recursive(tree, child, visited)
```

A Marcação:
O vértice é carimbado como descoberto (Vermelho) para evitar loops infinitos.

A Varredura:
O loop examina o leque de vizinhos adjacentes disponíveis.

O Mergulho:
A função suspende a si mesma e mergulha no próximo filho usando a Call Stack do sistema.

Gemini Notebook

Dissecando a Execução (Trace em Memória)

Rastreando a rota A → B → D



Passo 1 (Nó A)	Passo 2 (Mergulho B)	Passo 3 (Mergulho D)	Passo 4 (Backtracking D)
visited: {A} Stack: [dfs(A)]	visited: {A, B} Stack: [dfs(A) → dfs(B)]	visited: {A, B, D} Stack: [dfs(A) → dfs(B) → dfs(D)]	visited: {A, B, D} Stack: [dfs(A) → dfs(B)]

Gemini Notebook

DFS Iterativo: Assumindo o Controle






```python
def dfs_iterative(tree, start):
    visited = set()
    stack = [start]

    while stack:
        node = stack.pop()
        if node not in visited:
            visited.add(node)
            stack.extend(reversed(tree[node]))

```

Por que iterativo?

Em Python, o limite padrão de recursão é ~1000. Grafos massivos causarão um RecursionError fatal.

A Solução

Gerenciar uma estrutura de dados Pilha (Stack) explícita na memória heap do sistema.

Comportamento LIFO

O último elemento adicionado é o primeiro a ser explorado (pop), garantindo o mergulho agressivo.

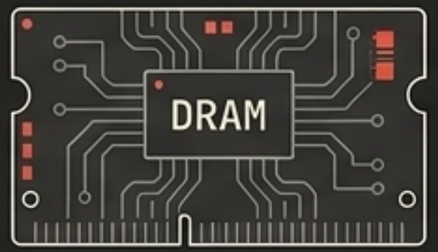
Gemini Notebook

O Custo da Exploração (Complexidade)



Tempo: O(V + E)

A função inicial examina cada Vértice (V) e cada Aresta (E) apenas uma vez. O custo computacional é estritamente linear ao tamanho da estrutura.



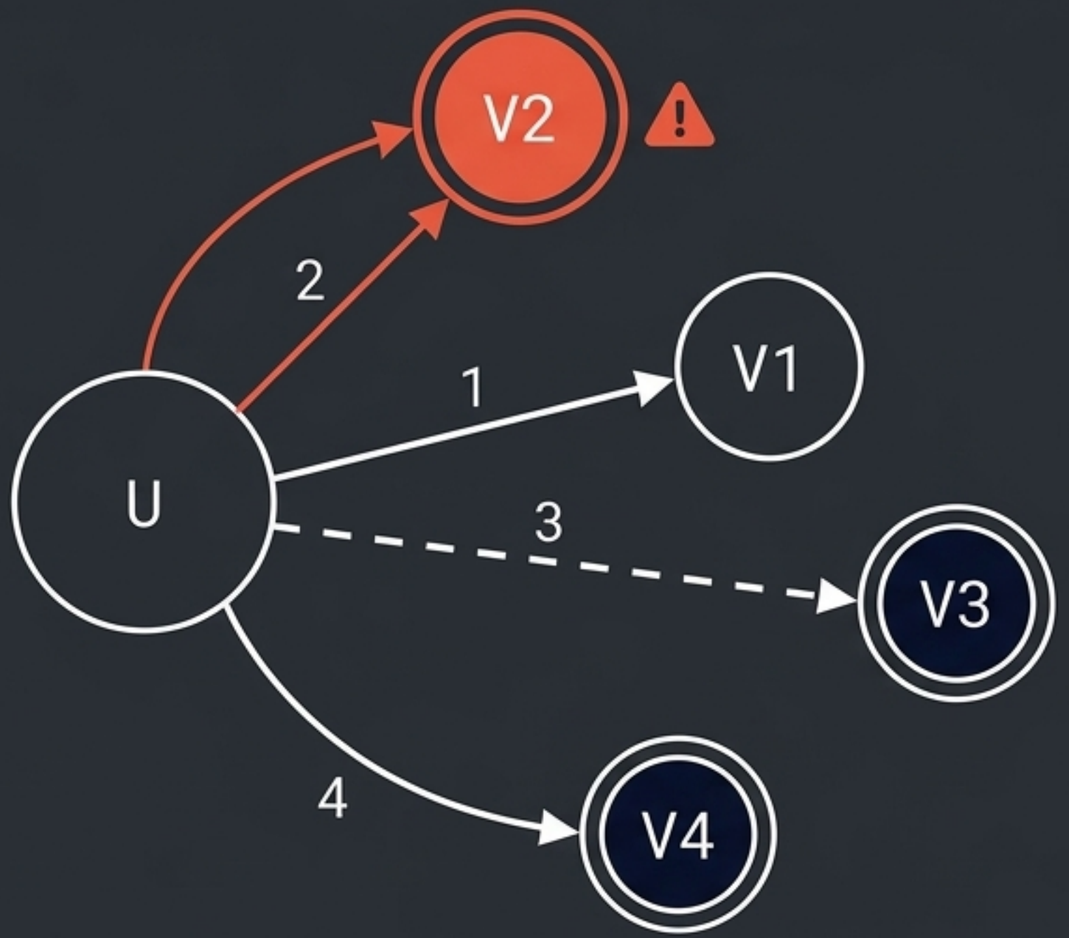
Espaço: O(V)

No pior cenário possível (um grafo formatado em linha reta), a pilha de recursão armazenará todos os Vértices (V) simultaneamente antes de conseguir conseguir fazer o primeiro backtracking.

Gemini Notebook

Dissecando o Terreno: Classificação de Arestas

O DFS atua como um scanner topográfico, categorizando todas as conexões em 4 tipos.



1. Aresta de Árvore
   Caminho normal da descoberta. Liga a um nó Branco.

2. Aresta de Retorno (Back Edge)
   Volta para um ancestral Vermelho. (ALERTA: Prova matematicamente a existência de CICLOS!)

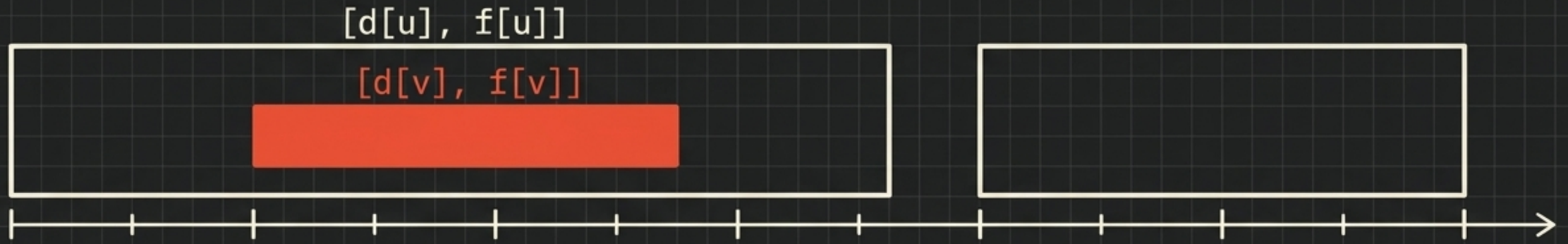
3. Aresta de Avanço (Forward Edge)
   Pula diretamente para um descendente já Preto.

4. Aresta de Cruzamento (Cross Edge)
   Pula lateralmente entre galhos diferentes já finalizados.

Gemini Notebook

O Teorema dos Parênteses

Os tempos de descoberta (d) e finalização (f) se comportam como tags HTML perfeitamente encaixadas.



- O tempo de vida de um descendente [d[v], f[v]] é inteiramente contido no de seu ancestral [d[u], f[u]].
- Se dois nós não são aparentados, seus intervalos de vida são totalmente disjuntos (um morre antes do outro nascer).
- Sobreposição parcial de intervalos é matematicamente impossível no algoritmo DFS.

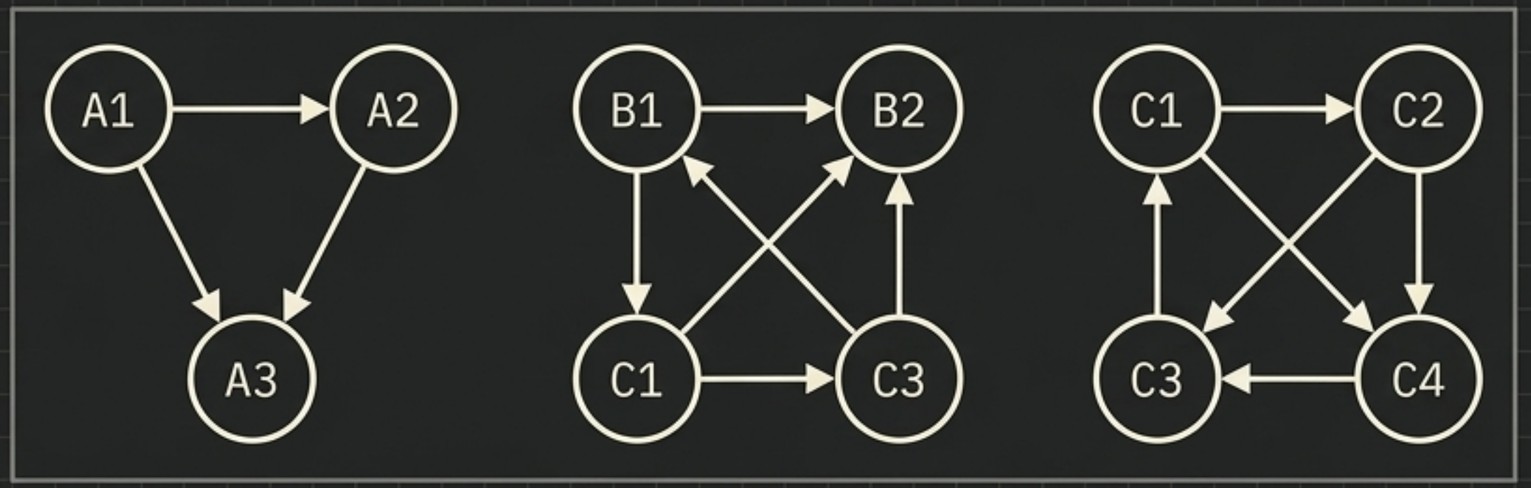
Gemini Notebook

Aplicação I: Contando Componentes Conexos

Como descobrir se um grafo gigante possui “ilhas” isoladas que não se comunicam?

Componentes: 3

Contagem Global



1. O loop global iterativo percorre todos os vértices da estrutura.
2. Sempre que encontrar um nó Branco intocado, uma “nova ilha” foi descoberta.
3. Incrementa-se o contador global (ncomps++).
4. O DFS é disparado a partir desse nó, espalhando-se e pintando toda a ilha atual de Preto.
5. O processo retoma a busca pelo próximo nó Branco.

Gemini Notebook

Aplicação II: Ordenação Topológica (O Conceito)

Modelando tarefas com dependências rígidas de precedência.

O Problema

Certas tarefas precisam ser concluídas antes de outras iniciarem (Sistemas de build, matrículas, cronogramas).

O Modelo (DAG)

Um Grafo Direcionado Acíclico. O fluxo é estritamente unidirecional, não existem ciclos lógicos.

O Objetivo

Esticar este grafo complexo em um arranjo linear garantindo que todas as setas sempre apontem para a direita.



Gemini Notebook

O Truque Mágico da Ordenação Topológica

Usando o timestamp de finalização para ordenar milhares de tarefas dependentes em tempo O(V+E).



Passo 1	Passo 2	Passo 3
Execute uma varredura DFS cega sobre todo o DAG.	Sempre que um nó for finalizado (cor Preta), insira-o no INÍCIO de uma lista ligada.	A lista resultante é a sua ordenação topológica perfeita.

Insight Teórico: Por que funciona? Porque quando um nó finaliza, o algoritmo garante absolutamente que todo o seu futuro já foi processado e já está posicionado à direita.

Gemini Notebook

Resumo e Folha de Cola (DFS Cheat Sheet)

QUANDO USAR?

- Detecção de Ciclos no código/sistema
- Ordenação Topológica (Dependências)
- Contar Componentes Conexos ('Ilhas')
- Solver labirintos / Backtracking

A MECÂNICA CORE

- A fronteira avança usando uma Pilha (LIFO).
- Pode ser Recursivo (Call Stack) ou Iterativo.
- Regra das Cores: Branco (Livre), Vermelho (Ativo/Na Pilha), Preto (Finalizado).

CUIDADOS (GOTCHAS)

- Loops Infinitos: Ocorre ao esquecer de marcar o Set de visitados.
- RecursionError: Grafos profundos estouram a pilha nativa do Python. Use iterativo!
- Falso Caminho: DFS acha UM caminho, mas nunca garante que é a rota mais curta.

COMPLEXIDADE (BIG-O)

- Tempo: O(V + E) (Processamento linear ultrarrápido).
- Espaço: O(V) no pior caso.
- Vantagem: Extremamente amigável à memória em comparação ao BFS para grafos muito amplos.

Gemini Notebook